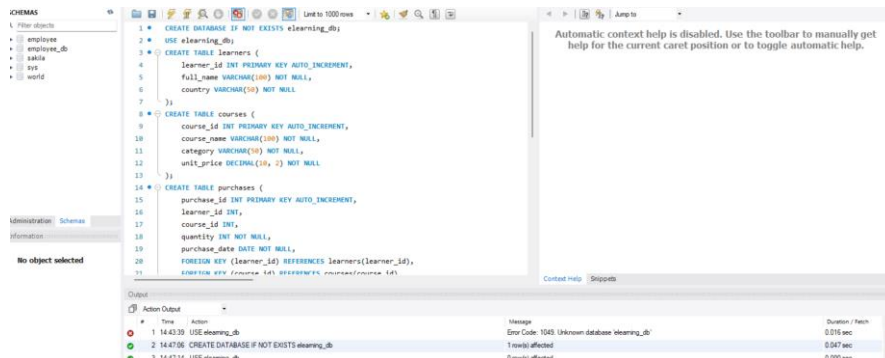


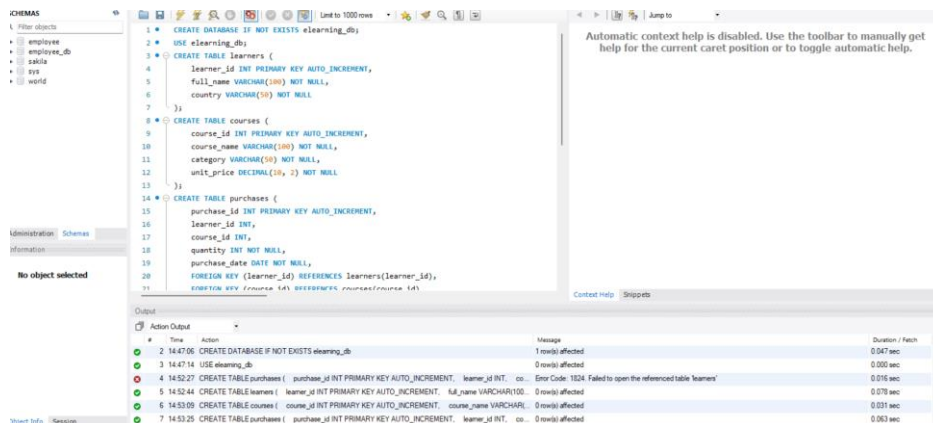
1. Database Setup & Data Entry

- Create a new database.



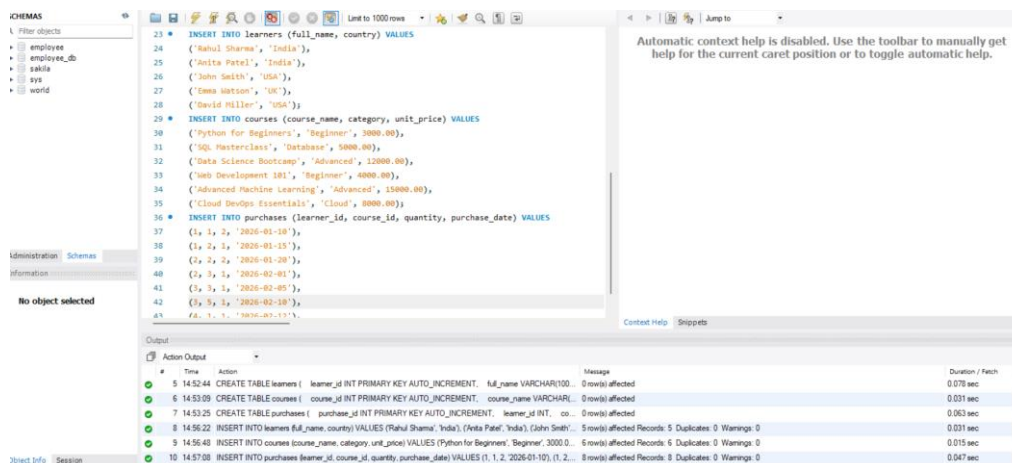
- Create the three tables with proper:

- Primary keys, Foreign keys, Appropriate data types



- Insert sample data:

- 4–5 learners
- 4–5 courses (different categories)
- 6–8 purchase records



2. Data Exploration Using Joins

Use INNER, LEFT, and RIGHT JOIN to:

- Combine learner, course, and purchase data
- Display: Learner name, Course name, Category, Quantity, Total amount, Purchase date
- Presentation Requirements:
 - Format currency to 2 decimal places
 - Use column aliases
 - Sort by the highest total amount

The screenshot shows a SQL IDE with a query editor on the left and an output window on the right. The query is as follows:

```
49 p.quantity AS Quantity,
50 FORMAT(p.quantity * c.unit_price, 2) AS Total_Amount,
51 p.purchase_date AS Purchase_Date
52 FROM purchases p
53 INNER JOIN learners l ON p.learner_id = l.learner_id
54 INNER JOIN courses c ON p.course_id = c.course_id
55 ORDER BY (p.quantity * c.unit_price) DESC;
56 *
57 1.full_name AS Learner_Name,
58 c.course_name AS Course_Name,
59 IFNULL(p.quantity, 0) AS Quantity
60 FROM learners l
61 LEFT JOIN purchases p ON l.learner_id = p.learner_id
62 LEFT JOIN courses c ON p.course_id = c.course_id;
63 *
64 c.course_name AS Course_Name,
65 p.purchase_id AS Purchase_ID,
66 IFNULL(p.quantity, 0) AS Quantity
67 FROM purchases p
68 RIGHT JOIN courses c ON p.course_id = c.course_id;
```

The output window shows a log of database actions, including INSERT and SELECT statements, with their respective durations and messages.

3. Core Analytical Queries (Q1–Q5)

Q1. Display each learner's total spending with their country.

The screenshot shows a SQL IDE with a query editor on the left and a result grid on the right. The query is as follows:

```
70 1.full_name,
71 1.country,
72 FORMAT(SUM(p.quantity * c.unit_price), 2) AS Total_Spending
73 FROM learners l
74 JOIN purchases p ON l.learner_id = p.learner_id
75 JOIN courses c ON p.course_id = c.course_id
76 GROUP BY l.learner_id, l.full_name, l.country;
```

The result grid displays the following data:

full_name	country	Total_Spending
Rahul Sharma	India	11,000.00
Arifa Patel	India	22,000.00
John Smith	USA	27,000.00
Emma Watson	UK	7,000.00

The output window shows a log of database actions, including INSERT and SELECT statements, with their respective durations and messages.

Q2. Find the top 3 most purchased courses by quantity.

The screenshot shows a SQL IDE interface with a query editor on the left and a results pane on the right. The query is as follows:

```

78 c.course_name,
79 SUM(p.quantity) AS Total_Quantity
80 FROM courses c
81 JOIN purchases p ON c.course_id = p.course_id
82 GROUP BY c.course_id, c.course_name
83 ORDER BY Total_Quantity DESC
84 LIMIT 3;

```

The results pane shows the following data:

course_name	Total_Quantity
Python for Beginners	3
SQL Masterclass	3
Data Science Bootcamp	2

The bottom pane shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
10	14:57:08	INSERT INTO purchases (learner_id, course_id, quantity, purchase_date) VALUES (1, 1, 2, '2026-01-10'), (1, ...	8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0	0.047 sec
11	15:03:20	SELECT l.full_name AS Learner_Name, c.course_name AS Course_Name, c.category AS Category, ...	8 row(s) returned	0.015 sec / 0.000 sec
12	15:04:05	SELECT l.full_name AS Learner_Name, c.course_name AS Course_Name, IFNULL(p.quantity, 0) AS ...	9 row(s) returned	0.016 sec / 0.000 sec
13	15:04:58	SELECT c.course_name AS Course_Name, p.purchase_id AS Purchase_ID, IFNULL(p.quantity, 0) AS ...	9 row(s) returned	0.000 sec / 0.000 sec
14	15:06:50	SELECT l.full_name, l.country, FORMAT(SUM(p.quantity * c.unit_price), 2) AS Total_Spending FROM ...	4 row(s) returned	0.031 sec / 0.000 sec
15	15:08:26	SELECT c.course_name, SUM(p.quantity) AS Total_Quantity FROM courses c JOIN purchases p ON c...	3 row(s) returned	0.000 sec / 0.000 sec

Q3. Show each category's:

- Total revenue
- Number of unique learners

The screenshot shows a SQL IDE interface with a query editor on the left and a results pane on the right. The query is as follows:

```

85 SELECT
86 c.category,
87 FORMAT(SUM(p.quantity * c.unit_price), 2) AS Total_Revenue,
88 COUNT(DISTINCT p.learner_id) AS Unique_Learners
89 FROM courses c
90 JOIN purchases p ON c.course_id = p.course_id
91 GROUP BY c.category;

```

The results pane shows the following data:

category	Total_Revenue	Unique_Learners
Advanced	39,000.00	2
Beginner	13,000.00	2
Database	15,000.00	2

The bottom pane shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
11	15:03:20	SELECT l.full_name AS Learner_Name, c.course_name AS Course_Name, c.category AS Category, ...	8 row(s) returned	0.015 sec / 0.000 sec
12	15:04:05	SELECT l.full_name AS Learner_Name, c.course_name AS Course_Name, IFNULL(p.quantity, 0) AS ...	9 row(s) returned	0.016 sec / 0.000 sec
13	15:04:58	SELECT c.course_name AS Course_Name, p.purchase_id AS Purchase_ID, IFNULL(p.quantity, 0) AS ...	9 row(s) returned	0.000 sec / 0.000 sec
14	15:06:50	SELECT l.full_name, l.country, FORMAT(SUM(p.quantity * c.unit_price), 2) AS Total_Spending FROM ...	4 row(s) returned	0.031 sec / 0.000 sec
15	15:08:26	SELECT c.course_name, SUM(p.quantity) AS Total_Quantity FROM courses c JOIN purchases p ON c...	3 row(s) returned	0.000 sec / 0.000 sec
16	15:10:00	SELECT c.category, FORMAT(SUM(p.quantity * c.unit_price), 2) AS Total_Revenue, COUNT(DISTIN...	3 row(s) returned	0.015 sec / 0.000 sec

Q4. List learners who purchased from more than one category.

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

```

92 SELECT
93     l.full_name
94 FROM learners l
95 JOIN purchases p ON l.learner_id = p.learner_id
96 JOIN courses c ON p.course_id = c.course_id
97 GROUP BY l.learner_id, l.full_name
98 HAVING COUNT(DISTINCT c.category) > 1;

```

Result Grid

Full_name
Rahul Sharma
Anita Patel

Output

#	Time	Action	Message	Duration / Fetch
12	15:04:08	SELECT l.full_name AS Learner_Name, c.course_name AS Course_Name, IFNULL(p.quantity, 0) AS	9 row(s) returned	0.016 sec / 0.000 sec
13	15:04:58	SELECT c.course_name AS Course_Name, p.purchase_id AS Purchase_ID, IFNULL(p.quantity, 0) AS	9 row(s) returned	0.000 sec / 0.000 sec
14	15:06:50	SELECT l.full_name, l.country, FORMAT(SUM(p.quantity * c.unit_price), 2) AS Total_Spending FROM	4 row(s) returned	0.031 sec / 0.000 sec
15	15:08:26	SELECT c.course_name, SUM(p.quantity) AS Total_Quantity FROM courses c JOIN purchases p ON c...	3 row(s) returned	0.000 sec / 0.000 sec
16	15:10:00	SELECT c.category, FORMAT(SUM(p.quantity * c.unit_price), 2) AS Total_Revenue, COUNT(DISTIN...	3 row(s) returned	0.015 sec / 0.000 sec
17	15:11:00	SELECT l.full_name FROM learners l JOIN purchases p ON l.learner_id = p.learner_id JOIN courses c ON p...	2 row(s) returned	0.000 sec / 0.000 sec

Q5. Identify courses never purchased.

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

```

98 HAVING COUNT(DISTINCT c.category) > 1;
99 SELECT
100     c.course_name,
101     c.category
102 FROM courses c
103 LEFT JOIN purchases p ON c.course_id = p.course_id
104 WHERE p.purchase_id IS NULL;

```

Result Grid

course_name	category
Cloud DevOps Essentials	Cloud

Output

#	Time	Action	Message	Duration / Fetch
13	15:04:58	SELECT c.course_name AS Course_Name, p.purchase_id AS Purchase_ID, IFNULL(p.quantity, 0) AS	9 row(s) returned	0.000 sec / 0.000 sec
14	15:06:50	SELECT l.full_name, l.country, FORMAT(SUM(p.quantity * c.unit_price), 2) AS Total_Spending FROM	4 row(s) returned	0.031 sec / 0.000 sec
15	15:08:26	SELECT c.course_name, SUM(p.quantity) AS Total_Quantity FROM courses c JOIN purchases p ON c...	3 row(s) returned	0.000 sec / 0.000 sec
16	15:10:00	SELECT c.category, FORMAT(SUM(p.quantity * c.unit_price), 2) AS Total_Revenue, COUNT(DISTIN...	3 row(s) returned	0.015 sec / 0.000 sec
17	15:11:00	SELECT l.full_name FROM learners l JOIN purchases p ON l.learner_id = p.learner_id JOIN courses c ON p...	2 row(s) returned	0.000 sec / 0.000 sec
18	15:12:07	SELECT c.course_name, c.category FROM courses c LEFT JOIN purchases p ON c.course_id = p.cour...	1 row(s) returned	0.000 sec / 0.000 sec

4. Subqueries & Correlated Subqueries

Q6. Find learners whose total spending is above the average learner spending.

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

```

114 FROM (
115     SELECT SUM(p2.quantity * c2.unit_price) AS learner_total
116     FROM purchases p2
117     JOIN courses c2 ON p2.course_id = c2.course_id
118     GROUP BY p2.learner_id
119 ) AS avg_table
120

```

Result Grid

Full_name	Total_Spending
Anita Patel	22000.00
John Smith	27000.00

Output

#	Time	Action	Message	Duration / Fetch
14	15:06:50	SELECT l.full_name, l.country, FORMAT(SUM(p.quantity * c.unit_price), 2) AS Total_Spending FROM	4 row(s) returned	0.031 sec / 0.000 sec
15	15:08:26	SELECT c.course_name, SUM(p.quantity) AS Total_Quantity FROM courses c JOIN purchases p ON c...	3 row(s) returned	0.000 sec / 0.000 sec
16	15:10:00	SELECT c.category, FORMAT(SUM(p.quantity * c.unit_price), 2) AS Total_Revenue, COUNT(DISTIN...	3 row(s) returned	0.015 sec / 0.000 sec
17	15:11:00	SELECT l.full_name FROM learners l JOIN purchases p ON l.learner_id = p.learner_id JOIN courses c ON p...	2 row(s) returned	0.000 sec / 0.000 sec
18	15:12:07	SELECT c.course_name, c.category FROM courses c LEFT JOIN purchases p ON c.course_id = p.cour...	1 row(s) returned	0.000 sec / 0.000 sec
19	15:14:07	SELECT l.full_name, SUM(p.quantity * c.unit_price) AS Total_Spending FROM learners l JOIN purchases...	2 row(s) returned	0.000 sec / 0.000 sec

Q7. Display courses whose price is higher than any course in the 'Beginner' category.

The screenshot shows a SQL IDE with a query editor on the left and a results pane on the right. The query is as follows:

```

123  unit_price
124  FROM courses
125  WHERE unit_price > ANY (
126    SELECT unit_price
127    FROM courses
128    WHERE category = 'Beginner'
129  )

```

The results pane shows a table with two columns: `course_name` and `unit_price`. The data is as follows:

course_name	unit_price
SQL Masterclass	9000.00
Data Science Bootcamp	12000.00
Web Development 101	4000.00
Advanced Machine Learning	15000.00
Cloud DevOps Essentials	8000.00

The bottom pane shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
16	15:08:26	SELECT	c.course_name, SUM(p.quantity) AS Total_Quantity FROM courses c JOIN purchases p ON c...	3 row(s) returned
17	15:10:00	SELECT	c.category, FORMAT(SUM(p.quantity * c.unit_price), 2) AS Total_Revenue, COUNT(DISTIN...	3 row(s) returned
18	15:11:00	SELECT	l.full_name FROM learners l JOIN purchases p ON l.learner_id = p.learner_id JOIN courses c ON p...	2 row(s) returned
19	15:12:07	SELECT	c.course_name, c.category FROM courses c LEFT JOIN purchases p ON c.course_id = p.cour...	1 row(s) returned
20	15:14:07	SELECT	l.full_name, SUM(p.quantity * c.unit_price) AS Total_Spending FROM learners l JOIN purchases...	2 row(s) returned
21	15:15:27	SELECT	course_name, unit_price FROM courses WHERE unit_price > ANY (SELECT unit_price ...	5 row(s) returned

Q8 . Find learners who spent more than the average spending in their country.

The screenshot shows a SQL IDE with a query editor on the left and a results pane on the right. The query is as follows:

```

142  FROM learners l2
143  JOIN purchases p2 ON l2.learner_id = p2.learner_id
144  JOIN courses c2 ON p2.course_id = c2.course_id
145  WHERE l2.country = l1.country
146  GROUP BY l2.learner_id
147  ) AS inner_spending
148  )

```

The results pane shows a table with three columns: `full_name`, `country`, and `Total_Spent`. The data is as follows:

full_name	country	Total_Spent
Anita Patel	India	22000.00

The bottom pane shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
16	15:10:00	SELECT	c.category, FORMAT(SUM(p.quantity * c.unit_price), 2) AS Total_Revenue, COUNT(DISTIN...	3 row(s) returned
17	15:11:00	SELECT	l.full_name FROM learners l JOIN purchases p ON l.learner_id = p.learner_id JOIN courses c ON p...	2 row(s) returned
18	15:12:07	SELECT	c.course_name, c.category FROM courses c LEFT JOIN purchases p ON c.course_id = p.cour...	1 row(s) returned
19	15:14:07	SELECT	l.full_name, SUM(p.quantity * c.unit_price) AS Total_Spending FROM learners l JOIN purchases...	2 row(s) returned
20	15:15:27	SELECT	course_name, unit_price FROM courses WHERE unit_price > ANY (SELECT unit_price ...	5 row(s) returned
21	15:16:30	SELECT	l1.full_name, l1.country, SUM(p1.quantity * c1.unit_price) AS Total_Spent FROM learners l1 J...	1 row(s) returned

5. CTE, CASE, View, and NULL Handling

Q9. Use a CTE to calculate total spending per learner, then:

Display learners with spending above 10,000.

CHEMAS

Filter objects

- employee
- employee_db
- skills
- sys
- world

Administration Schemas

Information

No object selected

```

173 GROUP BY l.Learner_id, l.full_name;
174 *
175 SELECT
176     c.course_name,
177     COALESCE(SUM(p.quantity), 0) AS Total_Purchased
178 FROM courses c
179 LEFT JOIN purchases p ON c.course_id = p.course_id
180 GROUP BY c.course_id, c.course_name;

```

Result Grid

course_name	Total_Purchased
Python for Beginners	3
SQL Masterclass	3
Data Science Bootcamp	2
Web Development 101	1
Advanced Machine Learning	1
Cloud DevOps Essentials	0

Output

Action Output

#	Time	Action	Message	Duration / Fetch
19	15:14:07	SELECT l.full_name, SUM(p.quantity * c.unit_price) AS Total_Spending FROM learners l JOIN purchases p ON l.Learner_id = p.Learner_id GROUP BY l.Learner_id, l.full_name	2 row(s) returned	0.000 sec / 0.000 sec
20	15:15:27	SELECT c.course_name, unit_price FROM courses WHERE unit_price > ANY (SELECT unit_price FROM courses)	5 row(s) returned	0.000 sec / 0.000 sec
21	15:16:30	SELECT l.full_name, l.country, SUM(p.quantity * c1.unit_price) AS Total_Spent FROM learners l JOIN purchases p ON l.Learner_id = p.Learner_id GROUP BY l.full_name, l.country	1 row(s) returned	0.015 sec / 0.000 sec
22	15:18:18	WITH LearnerSpending AS (SELECT l.Learner_id, l.full_name, SUM(p.quantity * c.unit_price) AS Total_Spent FROM learners l JOIN purchases p ON l.Learner_id = p.Learner_id GROUP BY l.Learner_id, l.full_name)	3 row(s) returned	0.000 sec / 0.000 sec
23	15:19:12	SELECT l.full_name, SUM(p.quantity * c.unit_price) AS Total_Spent, CASE WHEN SUM(p.quantity * c.unit_price) > 10000 THEN 'High Spender' ELSE 'Low Spender' END AS Spender_Type FROM learners l JOIN purchases p ON l.Learner_id = p.Learner_id GROUP BY l.full_name	4 row(s) returned	0.000 sec / 0.000 sec
24	15:20:50	SELECT c.course_name, COALESCE(SUM(p.quantity), 0) AS Total_Purchased FROM courses c LEFT JOIN purchases p ON c.course_id = p.course_id GROUP BY c.course_id, c.course_name	6 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

Q12 . View

- Create a view: category_performance_view
- Showing:
- Category
- Total revenue
- Number of purchases
- Average revenue per purchase

CHEMAS

Filter objects

- employee
- employee_db
- skills
- sys
- world

Administration Schemas

Information

No object selected

```

183 SUM(p.quantity * c.unit_price) AS Total_Revenue,
184 COUNT(p.purchase_id) AS Number_Of_Purchases,
185 AVG(p.quantity * c.unit_price) AS Avg_Revenue_Per_Purchase
186 FROM courses c
187 JOIN purchases p ON c.course_id = p.course_id
188 GROUP BY c.category;
189 *
190 SELECT * FROM category_performance_view;

```

Result Grid

Category	Total_Revenue	Number_Of_Purchases	Avg_Revenue_Per_Purchase
Beginner	13000.00	3	4333.333333
Database	15000.00	2	7500.000000
Advanced	39000.00	3	13000.000000

Output

Action Output

#	Time	Action	Message	Duration / Fetch
21	15:16:30	SELECT l.full_name, l.country, SUM(p.quantity * c1.unit_price) AS Total_Spent FROM learners l JOIN purchases p ON l.Learner_id = p.Learner_id GROUP BY l.full_name, l.country	1 row(s) returned	0.015 sec / 0.000 sec
22	15:18:18	WITH LearnerSpending AS (SELECT l.Learner_id, l.full_name, SUM(p.quantity * c.unit_price) AS Total_Spent FROM learners l JOIN purchases p ON l.Learner_id = p.Learner_id GROUP BY l.Learner_id, l.full_name)	3 row(s) returned	0.000 sec / 0.000 sec
23	15:19:12	SELECT l.full_name, SUM(p.quantity * c.unit_price) AS Total_Spent, CASE WHEN SUM(p.quantity * c.unit_price) > 10000 THEN 'High Spender' ELSE 'Low Spender' END AS Spender_Type FROM learners l JOIN purchases p ON l.Learner_id = p.Learner_id GROUP BY l.full_name	4 row(s) returned	0.000 sec / 0.000 sec
24	15:20:50	SELECT c.course_name, COALESCE(SUM(p.quantity), 0) AS Total_Purchased FROM courses c LEFT JOIN purchases p ON c.course_id = p.course_id GROUP BY c.course_id, c.course_name	6 row(s) returned	0.000 sec / 0.000 sec
25	15:22:55	CREATE OR REPLACE VIEW category_performance_view AS SELECT c.category AS Category, SUM(p.quantity * c.unit_price) AS Total_Revenue, COUNT(p.purchase_id) AS Number_Of_Purchases, AVG(p.quantity * c.unit_price) AS Avg_Revenue_Per_Purchase FROM courses c JOIN purchases p ON c.course_id = p.course_id GROUP BY c.category	0 row(s) affected	0.015 sec
26	15:23:21	SELECT * FROM category_performance_view LIMIT 0, 1000	3 row(s) returned	0.016 sec / 0.000 sec

Object Info Session